```
Web Crawl agent
# You may need to add your working directory to the Python path. To do so,
uncomment the following lines of code
# import sys
# sys.path.append("/Path/to/directory/agentic-framework") # Replace with
your directory path

import logging

from baf.core.agent import Agent
from baf.core.session import Session
from baf.exceptions.logger import logger
from baf.nlp.llm.llm_openai_api import LLMOpenAI
from baf.utils.web_crawl import crawl_website

# Configure the logging module (optional)
logger.setLevel(logging.INFO)

# Create the agent
agent = Agent('web_crawl_agent')  # set persist_sessions=True to enable
session persistence across restarts
# Load agent properties stored in a dedicated file
agent.load_properties('config.yaml')
# Define the platform your agent will use
# set authenticate_users=True to enable user authentication and previous
history loading in the UI
websocket_platform = agent.use_websocket_platform(use_ui=True)

# Create the LLM
gpt = LLMOpenAI(
    agent=agent,
    name='gpt-4o-mini',
    parameters={},
    num_previous_messages=10
)

webpages = crawl_website(
    initial_url="https://besser-pearl.org/",
    max_depth=3,
    max_pages=20,
    format="markdown",
    base_url_prefix="https://besser-pearl.org/"
)

# STATES

initial_state = agent.new_state('initial_state', initial=True)
awaiting_state = agent.new_state('awaiting_state')
answer_state = agent.new_state('answer_state')


# STATES BODIES' DEFINITION + TRANSITIONS
```

```
def initial_body(session: Session):
    session.reply('I am here to answer any questions you may have about the
BESSER project! Just say "hello" to get started.')


initial_state.set_body(initial_body)
initial_state.go_to(awaiting_state)

awaiting_state.when_no_intent_matched().go_to(answer_state)


def answer_body(session: Session):
    question = session.event.message
    system_message = \
    f"You are a helpful assistant for answering questions about the content
of a webpage. " \
    f"For any message not related to that, inform about what the user can
ask. " \
    f"Use the following webpage content to answer the question as best as
you can:\n{webpages}\n"
    answer = gpt.predict(message=question, system_message=system_message)
    session.reply(answer)
answer_state.set_body(answer_body)
answer_state.go_to(awaiting_state)


# RUN APPLICATION

if __name__ == '__main__':
    agent.run()
```